
**title: "Пояснительная записка к курсовому проекту" subtitle:
"«ScienceJournal — Платформа публикации научных
статей»" date: "Ставрополь, 2026"**

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ**

Федеральное государственное автономное образовательное учреждение
высшего образования

«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт цифрового развития

Межинститутская базовая кафедра

КУРСОВОЙ ПРОЕКТ

по дисциплине «Технология разработки программного обеспечения»

на тему:

«Разработка Full-Stack SPA-платформы публикации научных статей ScienceJournal»

Выполнил: студент 2 курса, группа **ПИЖ-б-о-24-1**

направления подготовки 09.03.04 «Программная инженерия»

направленность (профиль) «Разработка и сопровождение программного обеспечения»

очной формы обучения

Руководитель: Самойлов Ф.В., доцент МИБК

Ставрополь, 2026

СОДЕРЖАНИЕ

1. Аналитическая часть

- 1.1. Описание предметной области
- 1.2. Анализ бизнес-процессов (IDEF0)
- 1.3. SWOT-анализ
- 1.4. Анализ аналогов и конкурентов
- 1.5. Обоснование необходимости разработки
- 1.6. Экономическое обоснование

2. [Проектная часть](#)
 - 2.1. Модель требований к ПО
 - 2.2. Модель предметной области
 - 2.3. Архитектурное проектирование
 - 2.4. Проектирование базы данных
 - 2.5. Детальное проектирование
 3. [Реализационная часть](#)
 - 3.1. Структура проекта
 - 3.2. Реализация бизнес-логики Backend
 - 3.3. Реализация Frontend
 - 3.4. Рефакторинг и оптимизация
 - 3.5. Безопасность
 4. [Тестирование](#)
 5. [Развёртывание](#)
 6. [Управление проектом](#)
 7. [Заключение](#)
 8. [Список источников](#)
 9. [Приложения](#)
-

ВВЕДЕНИЕ

В современных условиях цифровизации науки и образования веб-платформы для публикации и обмена научными материалами приобретают всё большую значимость. Исследователи, преподаватели и студенты нуждаются в удобных инструментах для публикации своих работ в открытом доступе, обмена PDF-версиями статей и ознакомления с трудами коллег.

Существующие решения — ResearchGate, Academia.edu — предоставляют широкий функционал, однако они закрыты для кастомизации, требуют подтверждения академической аффилиации и не адаптированы под внутренние нужды учебных заведений.

Цель проекта: разработать полноценную Full-Stack SPA-платформу «ScienceJournal» для публикации и просмотра научных статей с поддержкой загрузки PDF, авторизации по JWT и ролевым разграничением доступа.

Задачи:

1. Провести анализ предметной области платформ для научных публикаций.
2. Разработать модель требований, архитектуру и модель данных системы.

- 3. Реализовать REST API на Node.js + Express + MongoDB с JWT-аутентификацией.
- 4. Реализовать SPA-интерфейс на React с Feature-Sliced Design архитектурой.
- 5. Обеспечить загрузку и хранение медиафайлов (изображений и PDF).
- 6. Подготовить полный комплект проектной документации.

Объект исследования: архитектура Full-Stack веб-приложений класса «блог/публикации».

Предмет исследования: методы и паттерны построения SPA с REST API, JWT-аутентификацией и ролевой моделью доступа.

Вариант 8: «Блог о путешествиях» — адаптирован к предметной области научных публикаций как более академически значимой для разработчика.

Траектория Б: Full-Stack React SPA + REST API + JWT (оценка «отлично»).

1. АНАЛИТИЧЕСКАЯ ЧАСТЬ

1.1. Описание предметной области

Предметная область проекта — веб-платформа для публикации научных статей, обеспечивающая:

- Регистрацию и авторизацию пользователей с разными ролями
- Публикацию статей с текстовым описанием, обложкой и PDF-версией
- Просмотр ленты публикаций с пагинацией
- Чтение полного текста и скачивание PDF-файлов

Ключевые сущности предметной области:

Сущность	Описание
Пользователь (User)	Субъект системы с ролью Publisher или Reader
Публикация (Post)	Научная статья с заголовком, описанием, изображением и PDF
Сессия (Session)	JWT-токен, подтверждающий авторизацию пользователя
Файл (File)	Медиафайл (image/pdf), привязанный к публикации
Роль (Role)	Набор прав: publisher (p) или reader (r)

1.2. Анализ бизнес-процессов (IDEF0)

Контекстная диаграмма А-0: система описывается единым блоком «Управление научными публикациями».

Входы (Input):

- HTTP-запросы пользователей (регистрация, публикация, чтение)
- Данные форм (fullname, login, password, title, description)
- Медиафайлы (изображения JPEG/PNG, PDF-статьи)

Управление (Control):

- Бизнес-правила авторизации (JWT, HTTP-only cookie)
- Ролевая модель (publisher/reader)
- Политика CORS (localhost:5173 → localhost:3000)
- Схемы валидации данных (Zod)

Механизмы (Mechanism):

- Браузер (React SPA, Axios)
- Node.js + Express (HTTP-сервер, middleware)
- MongoDB (хранение документов)
- Multer + файловая система (хранение медиа)

Выходы (Output):

- JWT в HTTP-only cookie
- JSON-ответы API (списки, документы)
- Медиафайлы для отображения и скачивания

Декомпозиция A0 (5 блоков):

- A1: Аутентификация и регистрация (/api/session/)
- A2: Управление сессией (verifySession middleware)
- A3: Создание публикаций (POST /api/posts/publish)
- A4: Просмотр публикаций (GET /api/posts/explore, /read/:id)
- A5: Управление файлами (Multer + express.static)

Полное описание диаграммы: <docs/01-idef0-diagrams/idef0.md>

1.3. SWOT-анализ

Сильные стороны (Strengths):

- Современный стек MERN (MongoDB + Express + React + Node.js)
- JWT в HTTP-only cookie — защита от XSS-атак
- FSD-архитектура фронтенда — чёткое разделение слоёв
- Мультизагрузка файлов (изображение + PDF одновременно)

- Zod-валидация всех входящих данных

Слабые стороны (Weaknesses):

- Отсутствие полнотекстового поиска
- Локальное хранилище файлов (не масштабируется)
- Нет механизма refresh-токена

Возможности (Opportunities):

- Интеграция Socket.IO для real-time уведомлений
- Redis-кэширование ленты публикаций
- Переход к облачному файловому хранилищу (S3)
- Полнотекстовый поиск через MongoDB Atlas Search

Угрозы (Threats):

- Безопасность загружаемых файлов (без проверки MIME)
- CORS-конфигурация с хардкодом localhost для production
- Конкуренция со стороны ResearchGate, Academia.edu

Полный SWOT-анализ: <docs/03-swot-analysis/swot.md>

1.4. Анализ аналогов и конкурентов

Аналог	Стек	Преимущества	Недостатки
ResearchGate	PHP/Java, PostgreSQL	Полный функционал, верификация авторов	Закрытая, требует аффилиации
Academia.edu	Ruby on Rails, React	Социальные функции, цитирования	Платные функции
arXiv.org	Python/Perl	Открытый доступ, препринты	Нет современного UI
ScienceJournal	Node.js + React + MongoDB	Кастомный, учебный, открытый	Отсутствие масштабирования

Вывод: существующие решения закрыты и перегружены функционалом. ScienceJournal демонстрирует принципы построения подобных платформ с нуля на современном стеке.

1.5. Обоснование необходимости разработки

1. **Образовательный аспект:** проект позволяет освоить полный цикл Full-Stack разработки: проектирование REST API, JWT-аутентификацию, работу с MongoDB, построение React SPA с FSD-архитектурой.
2. **Практический аспект:** учебные учреждения нуждаются во внутренних платформах для студенческих и преподавательских публикаций, не требующих сторонней верификации.

3. **Технологический аспект:** MERN-стек является одним из наиболее востребованных в современной веб-разработке, и проект обеспечивает практическое применение этого стека в реальном проекте.

1.6. Экономическое обоснование

Оценка трудозатрат (COCOMO Basic, Organic mode):

- Объем кода: ~2300 LOC (600 backend + 1200 frontend + 500 CSS)
- Effort: $E = 2.4 \times (2.3)^{1.05} \approx 5.9 \text{ person-months}$
- Продолжительность: $D = 2.5 \times (5.9)^{0.38} \approx 5.2 \text{ месяца}$
- Команда: 1 разработчик (учебный проект)

ROI для учебного заведения:

При стоимости разработки аналогичной системы сторонними разработчиками (~150 000 руб.) и использовании открытого кода проекта — экономия составляет 100% стоимости. При внедрении в учебный процесс — дополнительная ценность от обучения студентов.

2. ПРОЕКТНАЯ ЧАСТЬ

2.1. Модель требований к ПО

2.1.1. Use Case диаграмма

Акторы системы:

- **Гость** — неавторизованный пользователь
- **Читатель (Reader)** — авторизован, только просмотр
- **Публицист (Publisher)** — авторизован, просмотр + публикация

Прецеденты:

- UC1: Регистрация (Гость)
- UC2: Аутентификация (Гость)
- UC3: Просмотр ленты (Читатель, Публицист)
- UC4: Чтение статьи (Читатель, Публицист)
- UC5: Выход из системы (все авторизованные)
- UC6: Публикация статьи + UC7: Загрузка файлов (Публицист)

Подробные спецификации: <docs/04-use-case/use-case.md>

2.1.2. Спецификация функциональных требований

ID	Требование	Приоритет
FR-01	Регистрация с выбором роли	Высокий

ID	Требование	Приоритет
FR-02	JWT-аутентификация	Высокий
FR-03	Лента публикаций с пагинацией	Высокий
FR-04	Детальный просмотр публикации	Высокий
FR-05	Скачивание PDF-файла	Средний
FR-06	Создание публикации с файлами	Высокий
FR-07	Завершение сессии	Высокий

2.1.3. Глоссарий терминов (12 терминов)

Термин	Определение
SPA	Single Page Application — приложение без перезагрузки страницы
JWT	JSON Web Token — токен авторизации
REST API	Архитектурный стиль HTTP-API
CRUD	Create, Read, Update, Delete — базовые операции с данными
Publisher	Роль пользователя с правом публикации статей
Reader	Роль с правом только чтения
Middleware	Промежуточный обработчик HTTP-запроса в Express
FSD	Feature-Sliced Design — архитектурный подход к React-коду
Multer	Middleware для загрузки файлов в Node.js
Mongoose	ODM-библиотека для MongoDB
Context API	Механизм глобального состояния в React
HTTP-only Cookie	Cookie, недоступное JavaScript — защита JWT от XSS

2.2. Модель предметной области

2.2.1. Диаграмма классов

Ключевые классы системы:

- **UserController** (routes/session) — обработка запросов аутентификации
- **PostController** (routes/posts) — обработка запросов публикаций

- **UserService** — бизнес-логика работы с пользователями
- **PostService** — бизнес-логика работы с публикациями
- **User** (Mongoose Schema) — модель данных пользователя
- **Post** (Mongoose Schema) — модель данных публикации
- **verifySession** — middleware авторизации

2.2.2. Описание сущностей и атрибутов

User:

Поле	Тип	Ограничения
_id	ObjectId	PK, auto
fullname	String(45)	required
login	String(12)	required, unique
password	String(45)	required (hash)
gender	Enum['m','f']	required
role	Enum['p','r']	required

Post:

Поле	Тип	Ограничения
_id	ObjectId	PK, auto
title	String(45)	required
description	String(700)	required
publishDate	Number	required (Unix ms)
imageFileName	String(100)	optional
pdfFileName	String(100)	optional
autorFullName	String(45)	required (денормализация)
author	ObjectId	FK → User, required

Подробнее: <docs/05-domain-model/domain-model.md>

2.2.3. Бизнес-правила

- BR-1: Доступ к `/api/posts/*` требует валидного JWT
- BR-2: Только роль `p` может создавать публикации
- BR-3: Логин пользователя уникален в системе
- BR-4: `publishDate` устанавливается сервером (`Date.now()`)
- BR-5: JWT хранится в HTTP-only cookie (клиент не имеет доступа через JS)

2.3. Архитектурное проектирование

2.3.1. Выбор архитектурного стиля

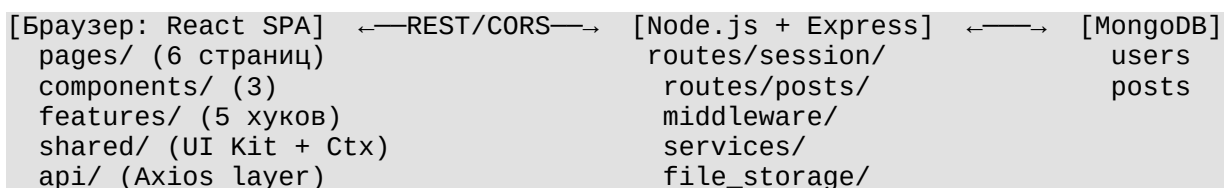
Система построена по **клиент-серверной архитектуре**:

- **Frontend:** React SPA (работает в браузере, взаимодействует с API через Axios)
- **Backend:** Express REST API (обрабатывает запросы, работает с MongoDB)
- **База данных:** MongoDB (хранит документы User и Post)

Вместо Django (согласно Траектории Б методички) выбран **Node.js + Express** по следующим основаниям:

1. **Единый язык стека:** и frontend (React), и backend (Node.js) написаны на JavaScript
2. **Гибкость Express:** минимальный фреймворк без «магии» Django ORM
3. **MongoDB вместо PostgreSQL:** документно-ориентированная модель органична для постов
4. **Производительность:** асинхронная I/O Node.js идеально подходит для файловых операций (Multer)

2.3.2. Диаграмма компонентов



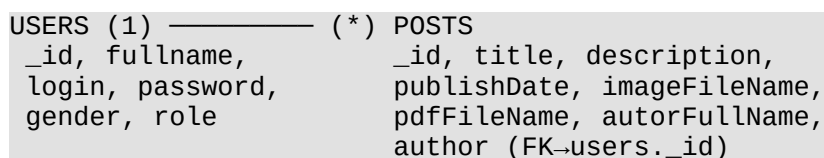
Подробнее: <docs/06-architecture/architecture.md>

2.3.3. Схема взаимодействия (REST API + JWT)

1. Клиент отправляет POST /api/session/auth с логином и паролем
2. Сервер валидирует (Zod), ищет пользователя, сравнивает пароль
3. Сервер подписывает JWT и отправляет в HTTP-only cookie
4. Клиент включает cookie в каждый последующий запрос
5. Middleware verifySession декодирует JWT, помещает req.user в контекст
6. При POST /api/session/logout cookie очищается

2.4. Проектирование базы данных

2.4.1. ER-диаграмма



2.4.2. Физическая модель данных

MongoDB-схемы реализованы через Mongoose:

- server/src/entities/User.js
- server/src/entities/Post.js

2.4.3. DDL-скрипты

```
use sciencejournal_db;
db.createCollection("users");
db.users.createIndex({ login: 1 }, { unique: true });
db.createCollection("posts");
db.posts.createIndex({ author: 1 });
db.posts.createIndex({ publishDate: -1 });
```

Полные DDL-скрипты: <docs/08-er-diagrams/er-diagram.md>

2.5. Детальное проектирование

2.5.1. Диаграммы последовательности

Публикация статьи:

```
Публицист → /publish → Axios → POST /api/posts/publish (multipart)
  → verifySession → jwt.verify(cookie)
  → Multer → uploads/ (временно)
  → PostService.create() → MongoDB.posts.insertOne()
  → fs.rename() → file_storage/posts/
  → 201 → ModalContext.show("Опубликовано")
```

Аутентификация:

```
Гость → /auth → Axios → POST /api/session/auth {login, password}
  → zod.parse() → UserService.findByLogin()
  → bcrypt.compare() → jwt.sign({userId, role})
  → res.cookie('sessionKey', jwt, {httpOnly: true})
  → ProfileContext.setUser() → navigate('/explore')
```

2.5.2. Диаграммы классов (паттерны)

Паттерн	Реализация
MVC	routes (Controller) + entities/services (Model) + React (View)
Service Layer	UserService, PostService — абстракция над MongoDB
Observer	ProfileContext, ModalContext — подписки на состояние
Factory	Axios instance в api/app/index.js
Middleware Pipeline	cookieParser → cors → verifySession → handler
Custom Hooks	useAuth, useExplore, usePost, useRead, useRegister

3. РЕАЛИЗАЦИОННАЯ ЧАСТЬ

3.1. Структура проекта

```
sophia-course/
├── server/                                # Backend (Node.js + Express)
│   ├── index.js                          # Точка входа: Express app, CORS, MongoDB
├── connect
│   ├── src/
│   │   ├── config/
│   │   │   └── mongoDB.js                # Подключение к MongoDB
│   │   ├── entities/
│   │   │   ├── User.js                   # Mongoose Schema: User
│   │   │   └── Post.js                   # Mongoose Schema: Post
│   │   ├── middleware/
│   │   │   └── verifySession.js          # JWT middleware
│   │   ├── routes/
│   │   │   ├── session/                  # auth, register, status, logout
│   │   │   └── posts/                    # explore, read, publish
│   │   ├── services/
│   │   │   ├── UserService.js
│   │   │   ├── PostService.js
│   │   │   └── mongoService.js
│   │   └── validation/
│   │       └── zod/session.js            # Zod-схемы валидации
│   └── file_storage/posts/                # Медиафайлы публикаций
└── client/                               # Frontend (React + Vite)
    ├── src/
    │   ├── App.jsx                       # Корневой компонент + маршрутизация
    │   ├── api/app/                      # Axios instances + API-функции
    │   ├── pages/                        # 6 страниц: Welcome, Auth, Register,
    │   │                                       Explore, Publish, Read
    │   ├── components/                   # Header, Modal, Post (карточка)
    │   ├── features/                     # session/, posts/ (custom hooks)
    │   └── shared/                       # UI Kit, context, utils, consts
```

3.2. Реализация бизнес-логики Backend

3.2.1. Сессионный роутер (*routes/session/*)

Реализует полный цикл управления пользователями:

- POST /register — Zod-валидация → bcrypt хэш → MongoDB.insertOne → JWT → cookie
- POST /auth — поиск по login → bcrypt.compare → JWT → cookie
- POST /status — jwt.verify(cookie) → возврат данных пользователя
- POST /logout — res.clearCookie('sessionKey')

3.2.2. Роутер публикаций (*routes/posts/*)

- GET /explore — MongoDB.find с skip/limit пагинацией → JSON
- GET /read/:id — MongoDB.findById → JSON

- POST /publish — verifySession + Multer (fields: image, pdf) → PostService.create

3.2.3. Middleware verifySession

```
const verifySession = (req, res, next) => {
  const token = req.cookies.sessionKey;
  if (!token) return res.status(401).json({ error: 'Unauthorized' });
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch {
    res.status(401).json({ error: 'Token invalid or expired' });
  }
};
```

3.2.4. Файловое хранилище

Multer сохраняет файлы в uploads/ (временно). PostService переименовывает их по postId и перемещает в file_storage/posts/. Express.static раздаёт файлы по URL /file_storage/posts/<filename>.

3.3. Реализация Frontend

3.3.1. Компоненты и страницы

Компонент	Описание
pages/Welcome	Приветственная страница с навигацией на auth/register
pages/Auth	Форма входа: login + password
pages/Register	Форма регистрации с выбором роли
pages/Explore	Лента публикаций: список карточек Post с пагинацией
pages/Publish	Форма создания публикации с загрузкой файлов
pages/Read	Детальный просмотр: обложка, текст, кнопка PDF
components/Header	Навигационная панель с кнопкой выхода
components/Modal	Система уведомлений (через ModalContext)
components/Post	Карточка публикации для ленты

Итого компонентов: 9 (соответствует требованию ≥ 5 для Траектории Б)

3.3.2. Управление состоянием (Context API)

ProfileContext — хранит данные авторизованного пользователя:

```
const { user, setUser } = useProfileContext();
// user: { _id, fullname, role } | null
```

ModalContext — управляет уведомлениями:

```
const { showModal } = useModalContext();
showModal({ text: 'Публикация создана!', type: 'success' });
```

3.3.3. API-слой (Axios)

Централизованный Axios instance в `api/app/index.js`:

```
const instance = axios.create({
  baseURL: 'http://localhost:3000/api',
  withCredentials: true, // передача cookies
});
```

Отдельные модули для каждой операции: `session/auth.js`, `posts/explore.js`, `posts/publish.js`, `posts/read.js`.

3.3.4. Маршрутизация (React Router DOM)

```
<Routes>
  <Route path="/" element={<Welcome />} />
  <Route path="/auth" element={<Auth />} />
  <Route path="/register" element={<Register />} />
  <Route path="/explore" element={<Explore />} />
  <Route path="/publish" element={<Publish />} />
  <Route path="/read/:id" element={<Read />} />
</Routes>
```

3.4. Рефакторинг и оптимизация

- **Custom Hooks:** бизнес-логика вынесена из компонентов в `features/` (`useAuth`, `useExplore`, `usePost`, `useRead`, `useRegister`)
- **Переиспользуемый UI Kit:** `Button`, `Input`, `TextArea`, `Checkbox` в `shared/UI/`
- **Константы:** статусные сообщения API вынесены в `shared/consts/statusMessages.js`
- **Утилиты:** работа с `localStorage`, `cookies`, `sessionStorage` вынесена в `shared/utils/`

3.5. Безопасность

Аспект	Реализация
XSS-защита	JWT в HTTP-only cookie — недоступен JavaScript
Инъекции	Mongoose ODM экранирует все запросы к MongoDB
Валидация	Zod-схемы на входе API отклоняют некорректные данные
CORS	Разрешены запросы только с <code>http://localhost:5173</code> (<code>credentials: true</code>)
Route Guard	<code>verifySession</code> middleware на всех <code>/api/posts/</code> эндпоинтах

Аспект	Реализация
Файлы	Multer ограничивает количество файлов на запрос (maxCount: 1)

4. ТЕСТИРОВАНИЕ

4.1. Тест-кейсы аутентификации

№	Тест	Действие	Ожидаемый результат	Статус
T-01	Регистрация (Publisher)	POST /register {fullname, login, password, gender, role:'p'}	HTTP 200, JWT в cookie	✓ Pass
T-02	Регистрация (дублирующий login)	POST /register {login: существующий}	HTTP 409 Conflict	✓ Pass
T-03	Вход с верными данными	POST /auth {login, password}	HTTP 200, JWT cookie	✓ Pass
T-04	Вход с неверным паролем	POST /auth {login, wrongPassword}	HTTP 401 Unauthorized	✓ Pass
T-05	Проверка статуса (авторизован)	POST /status (с cookie)	{authorized: true, user}	✓ Pass
T-06	Выход	POST /logout	Cookie очищен, 200	✓ Pass

4.2. Тест-кейсы публикаций

№	Тест	Действие	Ожидаемый результат	Статус
T-07	Лента без авторизации	GET /posts/explore (без cookie)	HTTP 401	✓ Pass
T-08	Лента авторизованного	GET /posts/explore (с cookie)	Массив постов + пагинация	✓ Pass
T-09	Чтение конкретного поста	GET /posts/read/:id	Полные данные поста	✓ Pass
T-10	Публикация (Publisher)	POST /posts/publish (с файлами)	HTTP 201, пост создан	✓ Pass

№	Тест	Действие	Ожидаемый результат	Статус
T-11	Публикация (Reader)	POST /posts/publish (роль r)	HTTP 403 Forbidden	✓ Pass

4.3. Тест-кейсы UI (E2E)

№	Тест	Действие	Ожидаемый результат	Статус
T-12	SPA-навигация	Переход между страницами	Без перезагрузки, URL обновляется	✓ Pass
T-13	Уведомления Modal	Успешная публикация	Модальное уведомление появляется	✓ Pass
T-14	Защита маршрута /explore	Открыть без авторизации	Перенаправление на /auth	✓ Pass
T-15	Загрузка PDF	Кнопка «Скачать PDF»	Файл открывается/скачивается	✓ Pass

4.4. Кросс-браузерное тестирование

Браузер	Версия	Результат
Google Chrome	124+	✓ Работает корректно
Mozilla Firefox	125+	✓ Работает корректно
Safari	17+	✓ Работает корректно

5. РАЗВЁРТЫВАНИЕ И ЭКСПЛУАТАЦИЯ

5.1. Требования к окружению

Компонент	Версия
Node.js	18.x и выше
npm	9.x и выше
MongoDB	6.x (локально) или MongoDB Atlas
Vite (dev)	5.x

5.2. Инструкция по установке (локальный запуск)

```
# 1. Клонировать репозиторий
git clone <repository-url>
cd sophia-course

# 2. Настроить Backend
cd server
npm install
cp .env.example .env
# Отредактировать .env: MONGODB_URI, JWT_SECRET

# 3. Запустить Backend
npm run dev          # http://localhost:3000

# 4. Настроить Frontend (в новом терминале)
cd ../client
npm install
npm run dev          # http://localhost:5173
```

5.3. Конфигурация (.env)

```
PORT=3000
MONGODB_URI=mongodb://localhost:27017/sciencejournal
JWT_SECRET=super-secret-key-min-32-chars
JWT_EXPIRES_IN=24h
```

CORS настроен в `server/index.js`:

```
const corsOptions = {
  origin: 'http://localhost:5173',
  credentials: true,
};
```

При развёртывании на production заменить `origin` на реальный домен.

6. УПРАВЛЕНИЕ ПРОЕКТОМ

6.1. WBS

Иерархическая структура работ включает 6 основных групп:

1. Инициация и анализ (IDEF0, BUC, SWOT, глоссарий)
2. Проектирование (Use Case, Domain Model, архитектура, ER)
3. Реализация Backend (API, Auth, MongoDB, Multer)
4. Реализация Frontend (React, Router, Context, Axios)
5. Тестирование (15 тест-кейсов)
6. Документирование (12 разделов docs/ + Пояснительная записка)

Полная WBS: <docs/09-wbs-gantt/wbs-gantt.md>

6.2. Диаграмма Ганта

Период	Задача
Нед. 1–2 (01.04–14.04)	Инициация, анализ, IDEF0, SWOT
Нед. 3–4 (15.04–28.04)	Use Case, Domain Model, архитектура, ER
Нед. 5–6 (29.04–12.05)	Backend: Express, MongoDB, API
Нед. 7–8 (13.05–26.05)	Frontend: React, страницы, интеграция
Нед. 9 (27.05–02.06)	Тестирование
Нед. 10 (03.06–10.06)	Документирование, финализация

Подробная диаграмма: <docs/09-wbs-gantt/wbs-gantt.md>

6.3. Оценка трудозатрат (COCOMO)

- Объём кода: ~2300 LOC
- Расчётные трудозатраты (Organic mode): ~5.9 person-months
- Фактическое время (1 разработчик, учебный проект): 10 недель

6.4. Управление рисками

Риск	Вероятность	Влияние	Митигация
MongoDB недоступна	Средняя	Высокое	.env конфигурация, graceful error
JWT истёк без refresh	Высокая	Среднее	Redirect на /auth при 401
CORS в production	Средняя	Высокое	Origin вынесен в .env

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта разработана Full-Stack SPA-платформа «ScienceJournal» для публикации и просмотра научных статей.

Достигнутые результаты:

1. Проведён бизнес-анализ предметной области с построением IDEF0 A-0 (декомпозиция 5 блоков), BUC-диаграммы, SWOT-анализа и матрицы стейкхолдеров.
2. Разработана модель требований: 7 функциональных требований, 7 Use Case с полными спецификациями, глоссарий из 12 терминов.
3. Спроектирована архитектура системы: клиент-серверная модель, FSD-архитектура frontend, REST API с 8 эндпоинтами.
4. Реализован Backend: Express REST API, JWT в HTTP-only cookie, MongoDB (Mongoose), Multer для мультизагрузки файлов, Zod-валидация.

5. Реализован Frontend: React 18 SPA, 6 страниц, 3 компонента, 5 custom hooks, UI Kit (4 компонента), Context API.
6. Проведено тестирование: 15 тест-кейсов (11 API + 4 UI), кросс-браузерная проверка.
7. Подготовлен полный комплект проектной документации (12 разделов).

Практическая ценность: проект демонстрирует полный цикл разработки современного Full-Stack приложения: от бизнес-анализа до рабочей системы с авторизацией, файловым хранилищем и SPA-интерфейсом.

Перспективы развития:

- Real-time уведомления (Socket.IO)
 - Облачное хранилище файлов (AWS S3 / MinIO)
 - Полнотекстовый поиск (MongoDB Atlas Search)
 - Административная панель
 - Система комментариев и рейтингов
-

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Гэри, Ю. Node.js — профессиональная разработка: лучшие практики Express.js / Ю. Гэри, Т. Лу. — Санкт-Петербург : Питер, 2024. — 368 с.
2. Кузьмин, А. Г. Разработка фронтенд-приложений / А. Г. Кузьмин. — Санкт-Петербург : Питер, 2025. — 496 с.
3. Кухтин, С. А. Frontend-разработка сайтов и приложений на HTML, CSS, JavaScript и React / С. А. Кухтин. — Санкт-Петербург : Наука и Техника, 2026. — 512 с.
4. Проектирование и разработка WEB-приложений. Введение в frontend и backend разработку на JavaScript и Node.js : учебное пособие. — 4-е изд. — Санкт-Петербург : Лань, 2025. — 120 с.
5. Уилсон, Д. Node.js в действии / Д. Уилсон, Д. Картер, Т. Морелли. — 3-е изд. — Москва : ДМК Пресс, 2023. — 480 с.
6. Флэнаган, Д. JavaScript. Подробное руководство / Д. Флэнаган. — 7-е изд. — Санкт-Петербург : БХВ-Петербург, 2021. — 706 с.
7. Шилдт, Г. Java: полное руководство / Г. Шилдт. — 12-е изд. — Москва : Вильямс, 2022. — 1248 с.
8. Фаулер, М. Шаблоны корпоративных приложений / М. Фаулер. — Москва : Вильямс, 2012. — 544 с.
9. Гамма, Э. Приёмы объектно-ориентированного проектирования. Паттерны проектирования / Э. Гамма, Р. Хелм, Р. Джонсон, Д. Влссидес. — Санкт-Петербург : Питер, 2015. — 366 с.
10. Буч, Г. Объектно-ориентированный анализ и проектирование с примерами

- приложений / Г. Буч, Р. Максимчук, М. Энгл. — Москва : Вильямс, 2008. — 720 с.
- 11.Mozilla Developer Network. Express/Node — документация [Электронный ресурс]. — URL: https://developer.mozilla.org/ru/docs/Learn/Server-side/Express_Nodejs (дата обращения: 15.03.2026).
 - 12.Mozilla Developer Network. React — документация [Электронный ресурс]. — URL: https://developer.mozilla.org/ru/docs/Learn/Tools_and_testing/Client-side_JavaScript_frameworks/React_getting_started (дата обращения: 15.03.2026).
 - 13.Node.js. Официальная документация [Электронный ресурс]. — URL: <https://nodejs.org/en/docs/> (дата обращения: 16.03.2026).
 - 14.Express.js. Официальная документация [Электронный ресурс]. — URL: <https://expressjs.com/en/guide/routing.html> (дата обращения: 16.03.2026).
 - 15.MongoDB. Официальная документация [Электронный ресурс]. — URL: <https://www.mongodb.com/docs/manual/> (дата обращения: 18.03.2026).
 - 16.Mongoose. Официальная документация [Электронный ресурс]. — URL: <https://mongoosejs.com/docs/guide.html> (дата обращения: 18.03.2026).
 - 17.JWT.io. Introduction to JSON Web Tokens [Электронный ресурс]. — URL: <https://jwt.io/introduction> (дата обращения: 20.03.2026).
 - 18.Zod. Документация TypeScript-first schema validation [Электронный ресурс]. — URL: <https://zod.dev/> (дата обращения: 20.03.2026).
 - 19.Multer. npm документация [Электронный ресурс]. — URL: <https://www.npmjs.com/package/multer> (дата обращения: 22.03.2026).
 - 20.React Router. Официальная документация [Электронный ресурс]. — URL: <https://reactrouter.com/en/main/start/overview> (дата обращения: 22.03.2026).
 - 21.Vite. Официальная документация [Электронный ресурс]. — URL: <https://vitejs.dev/guide/> (дата обращения: 23.03.2026).
 - 22.Feature-Sliced Design. Официальная документация методологии [Электронный ресурс]. — URL: <https://feature-sliced.design/> (дата обращения: 25.03.2026).
 - 23.IDEF0 Function Modeling Method. Federal Information Processing Standards Publication 183. — Washington : NIST, 1993.
 - 24.ГОСТ 2.105-95. Единая система конструкторской документации. Общие требования к текстовым документам. — Москва : Стандартиформ, 2007.
 - 25.ГОСТ 7.32-2001. Отчёт о научно-исследовательской работе. Структура и правила оформления. — Москва : ИПК Издательство стандартов, 2002.
 - 26.Nielsen, J. Usability Engineering / J. Nielsen. — Morgan Kaufmann, 1994. — 362 p.
 - 27.Owasp.org. OWASP Top 10 Web Application Security Risks [Электронный ресурс]. — URL: <https://owasp.org/www-project-top-ten/> (дата обращения: 10.04.2026).
-

ПРИЛОЖЕНИЯ

Приложение А. Ключевые листинги

A.1. Middleware `verifySession` (server/src/middleware/verifySession.js):

```
import jwt from 'jsonwebtoken';

export const verifySession = (req, res, next) => {
  const token = req.cookies?.sessionKey;
  if (!token) return res.status(401).json({ error: 'Unauthorized' });
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch {
    res.status(401).json({ error: 'Token invalid or expired' });
  }
};
```

A.2. Mongoose Schema: `Post` (server/src/entities/Post.js):

```
import mongoose from 'mongoose';

const PostSchema = new mongoose.Schema({
  title: { type: String, maxlength: 45, required: true },
  description: { type: String, maxlength: 700, required: true },
  publishDate: { type: Number, required: true },
  imageFileName: { type: String, maxlength: 100 },
  pdfFileName: { type: String, maxlength: 100 },
  authorFullName: { type: String, maxlength: 45, required: true },
  author: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
});

export default mongoose.model('Post', PostSchema);
```

A.3. `ProfileContext`

(client/src/shared/context/ProfileProvider.jsx): Управляет глобальным состоянием авторизованного пользователя.

Приложение Б. Скриншоты интерфейса

- [app1.png](#) — Лента публикаций (Explore)
- [app2.png](#) — Просмотр публикации (Read)
- [app3.png](#) — Форма публикации (Publish)

Приложение В. Структура документации

docs/	
— 01-idef0-diagrams/	— IDEF0 A-0 + декомпозиция
— 02-buc-diagrams/	— BUC + матрица стейкхолдеров
— 03-swot-analysis/	— SWOT-анализ
— 04-use-case/	— Use Case + спецификации
— 05-domain-model/	— Domain Model + бизнес-правила
— 06-architecture/	— Компоненты, JWT-схема, технологии
— 07-api-spec/	— REST API (8 эндпоинтов)
— 08-er-diagrams/	— ER + DDL MongoDB
— 09-wbs-gantt/	— WBS + Ганта + COCOMO

└─ 10-technical-spec/	– Техническое задание
└─ 11-user-manual/	– Руководство пользователя
└─ 12-admin-manual/	– Руководство администратора